Loan Application Analyzer

Lakshmi Pr

Anees Shaik

Rachana Muvva

Monisha Udutha

Vamshi Ch

Table of Content

- Data description
- Data Features
- Scatter plot
- Analyzed Features
- Histogram, Density, and box plots
- Interpretation of plots
- Codes
- Confusion matrix
- Graphs for linear equation, y =aX+b
- Interpretation of resluts

Data description

→The Loan Application Analyzer collects and analyzes applicant data, assessing creditworthiness and risk factors to facilitate informed lending decisions.

→ https://github.com/prasertcbs/basic-dataset/blob/master/Loan-Approval-Prediction.csv

Data Features

→ Loan ID

→ Gender

→ Married status

→ Education

→ Employed status

→ Applicant / co-applicant Income

→ Loan amount

→ Credit History

→ Duration of the loan

→ Residence Area

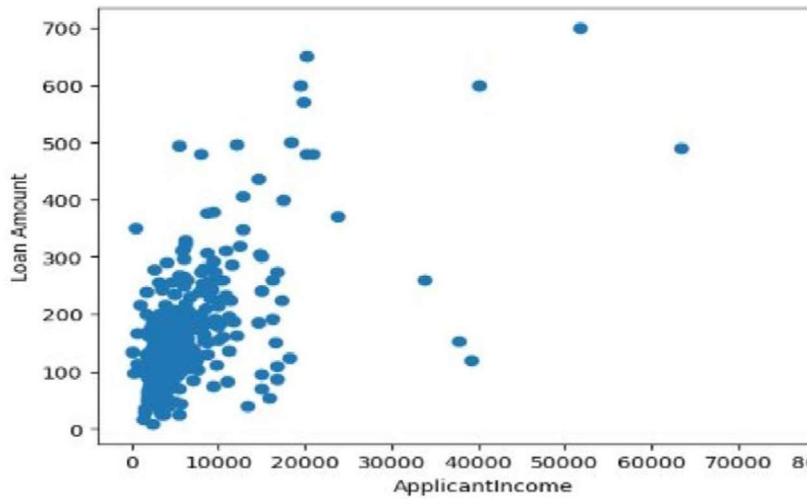
→ Loan Status

Scatter plot

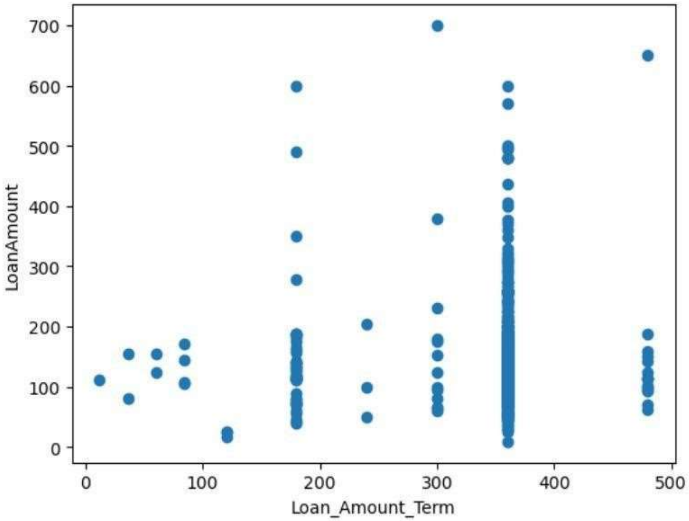
Scatter plot of quantitative columns:

- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount term

```
import matplotlib.pyplot as plt
x = df.ApplicantIncome
y = df.LoanAmount
plt.xlabel("ApplicantIncome")
plt.ylabel("Loan Amount")
plt.scatter(x, y)
plt.show()
```




```
import matplotlib.pyplot as plt
x = df.Loan_Amount_Term
y = df.LoanAmount
plt.xlabel("Loan_Amount_Term")
plt.ylabel("LoanAmount")
plt.scatter(x, y)
plt.show()
```



Analyzed Features

Quantitative Features:

- Applicant Income
- Co Applicant Income ● Loan Amount
- Loan Amount tenure

Qualitative Features:

- Married
- Self Employed

Inserting Features

```
df1 = df.filter([ 'ApplicantIncome', 'CoapplicantIncome',  'LoanAmount',  'Loan_Amount_Term'], axis=1)
```

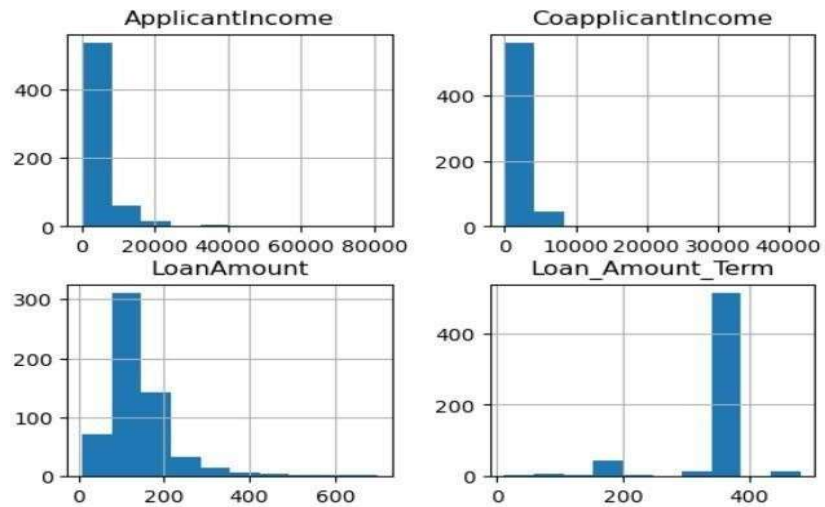
```
df1
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term
0	5849	0.0	NaN	360.0
1	4583	1508.0	128.0	360.0
2	3000	0.0	66.0	360.0
3	2583	2358.0	120.0	360.0
4	6000	0.0	141.0	360.0
...	...	...	...	...
609	2900	0.0	71.0	360.0
610	4106	0.0	40.0	180.0
611	8072	240.0	253.0	360.0
612	7583	0.0	187.0	360.0
613	4583	0.0	133.0	360.0

614 rows × 4 columns

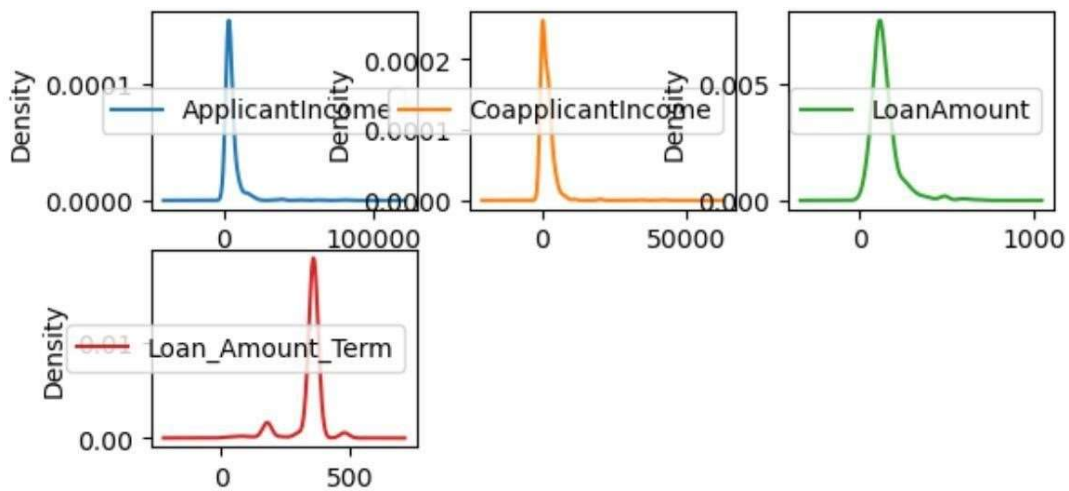
Histogram Plots

```
import matplotlib.pyplot as plt
df1.hist()
plt.show()
```



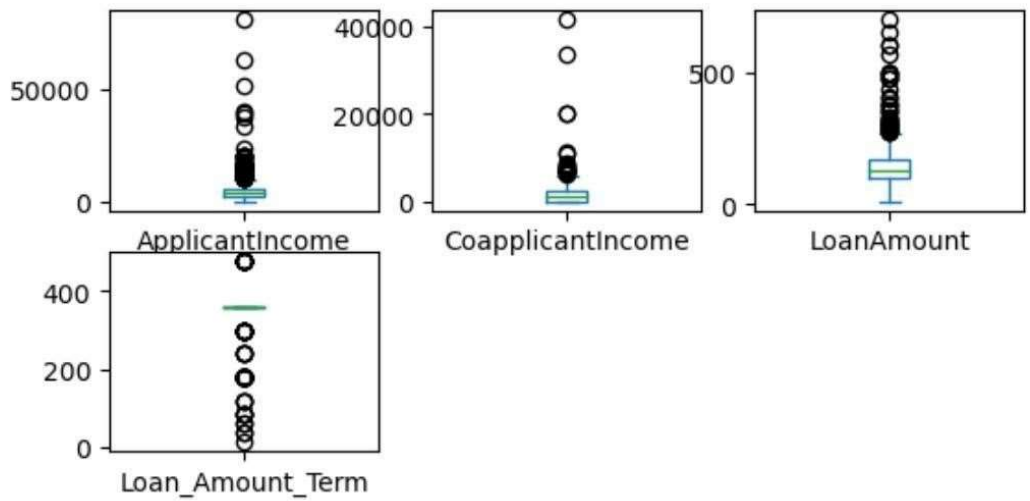
Density Plots

```
df1.plot(kind='density', subplots = True, layout=(3,3), sharex=False)
plt.show()
```



Box plots

```
df1.plot(kind='box', subplots = True, layout=(3,3), sharey=False)
plt.show()
```



Interpretation of plots

- Scatter plots show the relationship between two quantitative variables. In this case, we're visualizing the relationship between "ApplicantIncome" and “Loan amount“ & “Loan amount” and “Loan tenure”.
- The histogram provides a visual representation of the distribution of a single variable, in this case, "Applicant Income“, “loan amount”, loan_amount_term” and "Co applicant Income.“
- The Density plot is similar to a histogram but provides a smoothed representation of the data. The density plot for "Loan Amount" provides a smoothed representation of the distribution of loan amounts requested by applicants. It confirms that there is a peak in the density plot around a specific loan amount, indicating a common loan amount that many applicants request.
- Box plots display the distribution of data and identify potential outliers.

Codes

```python
#loading the dataset
data = pd.read_csv("Loan-Approval-Prediction.csv")

#handling missing values in 'LoanAmount'
data['LoanAmount'].fillna(data['LoanAmount'].median(), inplace=True)

#selecting the two attributes
X = data['ApplicantIncome'].values.reshape(-1, 1)
y = data['LoanAmount'].values.reshape(-1, 1)

#a linear regression model
model = LinearRegression()
model.fit(X, y)

#predictions using the model
y_pred = model.predict(X)

#Plot the linear regression line
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, y_pred, color='red', linewidth=2, label='Linear Regression Line')
plt.xlabel('ApplicantIncome')
plt.ylabel('LoanAmount')
plt.legend()
plt.title('Linear Regression: LoanAmount vs. ApplicantIncome')
plt.show()
```

```python
X = data[['ApplicantIncome']]
# Convert 'Y' to 1, 'N' to 0
y = (data['Loan_Status'] == 'Y').astype(int)

# Create a logistic regression model
model = LogisticRegression(solver='liblinear')

# Fit the model to the data
model.fit(X, y)

# Make predictions on the same data
y_pred = model.predict(X)

# Calculate the confusion matrix
conf_matrix = confusion_matrix(y, y_pred)
print("Confusion Matrix:")
print(conf_matrix)

# Calculate the accuracy score
accuracy = accuracy_score(y, y_pred)
print("\nAccuracy Score:", accuracy)
```

Confusion matrix

Confusion matrix: [[0 192] [0 422]]

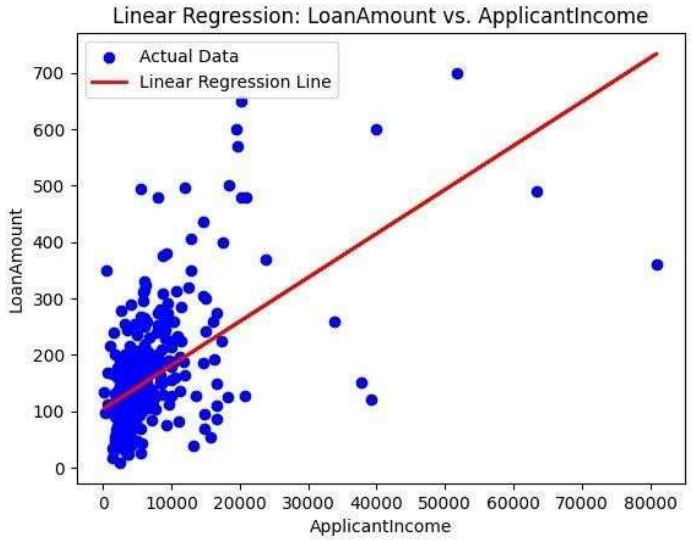
- The top-left cell (0,0) represents the True Negatives (TN):
  This is the number of instances that were correctly predicted as the negative class ('N' or 0). According to the results, it's 0.

- The top-right cell (0,1) represents the False Positives (FP):
  This is the number of instances that were incorrectly predicted as the positive class ('Y' or 1) when they were actually the negative class. According to the results, it's 192.

- The bottom-left cell (1,0) represents the False Negatives (FN):
  This is the number of instances that were incorrectly predicted as the negative class ('N' or 0) when they were actually the positive class. According to the results, it's 0.

- The bottom-right cell (1,1) represents the True Positives (TP):
  This is the number of instances that were correctly predicted as the positive class ('Y' or 1). According to the results, it's 422.

Graph of linear regression, y=aX+b



Linear Equation: y = -5.223012366664114e-06X + 0.84153322825429

Interpretation

- The accuracy score is approximately 0.6873 or 68.73%. This means that the model correctly predicted the class labels for about 68.73% of the instances in the dataset.

- The model achieved a relatively high accuracy score, indicating that it correctly predicted the 'Y' and 'N' labels for a significant portion of the data.

- But, It seems that there is an imbalance, with a large number of instances in the 'Y' class (422) and no instances correctly predicted in the 'N' class.

- The model did not correctly predict any instances as 'N', which is a concern and may indicate that the model has a bias toward predicting the positive class.

Decision Tree Code

```python
import pandas
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt
df = pandas.read_csv("Loan-Approval-Prediction.csv")
print(df)
```

```
    Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001002   Male      No          0      Graduate            No
1  LP001003   Male     Yes          1      Graduate            No
2  LP001005   Male     Yes          0      Graduate           Yes
3  LP001006   Male     Yes          0  Not Graduate            No
4  LP001008   Male      No          0      Graduate            No
5  LP001011   Male     Yes          2      Graduate           Yes
6  LP001013   Male     Yes          0  Not Graduate            No
7  LP001014   Male     Yes         3+      Graduate            No
8  LP001018   Male     Yes          2      Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             5849                  0         NaN               360
1             4583               1508       128.0               360
2             3000                  0        66.0               360
3             2583               2358       120.0               360
4             6000                  0       141.0               360
5             5417               4196       267.0               360
6             2333               1516        95.0               360
7             3036               2504       158.0               360
8             4006               1526       168.0               360

   Credit_History Property_Area Loan_Status
0               1         Urban           Y
1               1         Rural           N
2               1         Urban           Y
3               1         Urban           Y
4               1         Urban           Y
5               1         Urban           Y
6               1         Urban           Y
7               0     Semiurban           N
8               1         Urban           Y
```

```python
X = df[['LoanAmount', 'Loan_Amount_Term']]
y = df[['ApplicantIncome']]
```

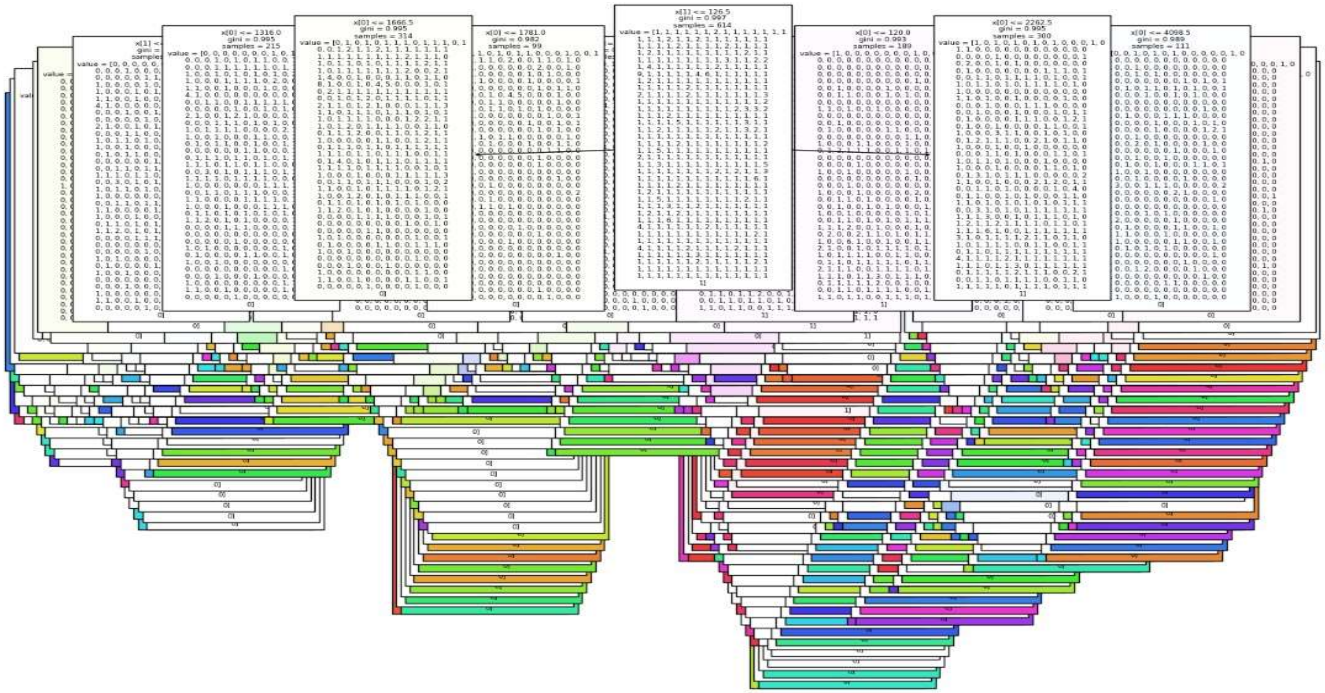
```python
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20)
```

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
clf = DecisionTreeClassifier()
clf.fit(X_train,y_train)
clf.predict(X_test)
print(accuracy_score(y_true=y_train,y_pred=clf.predict(X_train)))
print(accuracy_score(y_true=y_test,y_pred=clf.predict(X_test)))
```

```
0.8571428571428571
0.0
```

```python
from sklearn.tree import plot_tree

plt.figure(figsize=(16, 12))
plot_tree(clf, filled=True, feature_names=X.columns.tolist(), class_names=list(map(str, df['ApplicantIncome'])), rounded=True, fontsize=10)
plt.show()
```

Results:



Since the dataset is exceptionally large, it can be challenging to draw meaningful conclusions from the decision tree. To address this issue, we have reduced the dataset to a more manageable size, selecting, for example, only 10 rows, to facilitate more informed decision-making.

Code after reducing the dataset:

```
X = df[['LoanAmount', 'Loan_Amount_Term']]
y = df[['ApplicantIncome']]
```

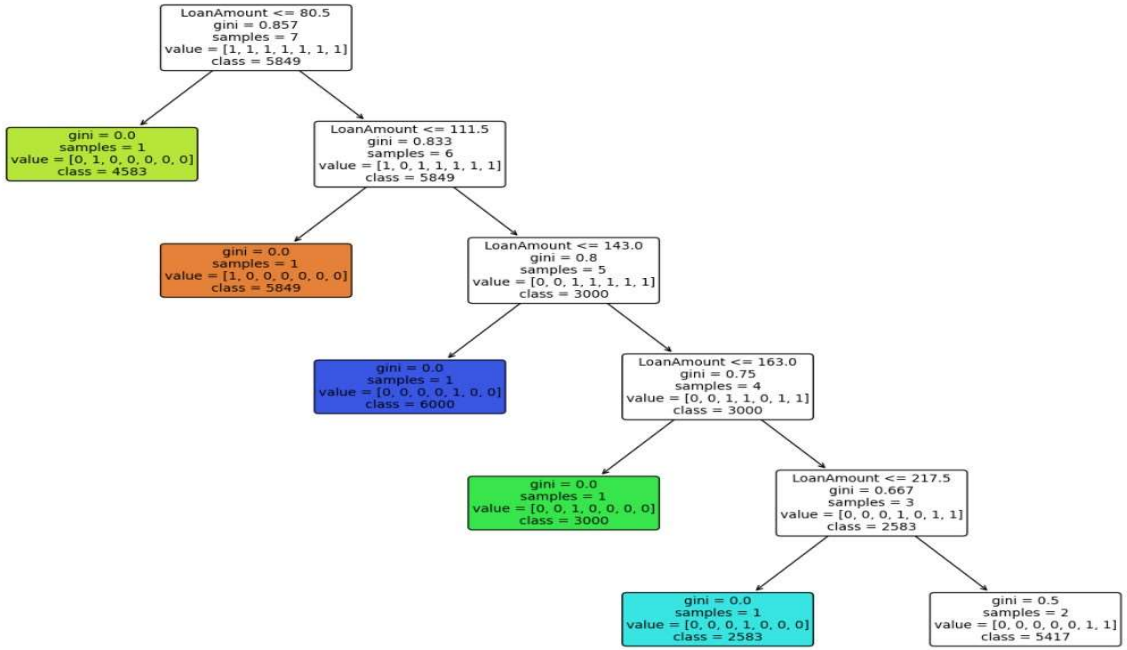
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20)
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
clf = DecisionTreeClassifier()
clf.fit(X_train,y_train)
clf.predict(X_test)
print(accuracy_score(y_true=y_train,y_pred=clf.predict(X_train)))
print(accuracy_score(y_true=y_test,y_pred=clf.predict(X_test)))
```

```
0.8571428571428571
0.0
```

```
from sklearn.tree import plot_tree

plt.figure(figsize=(16, 12))
plot_tree(clf, filled=True, feature_names=X.columns.tolist(), class_names=list(map(str, df['ApplicantIncome'])), rounded=True, fontsize=10)
plt.show()
```

Decision Tree



Interpretation

The tree starts with a root node that decides based on one of the features ('ApplicantIncome,' 'LoanAmount) to split the data into two or more branches.

Each subsequent node in the tree represents a decision based on a feature.

The leaf nodes of the tree represent the predicted class labels for the instances that reach them.

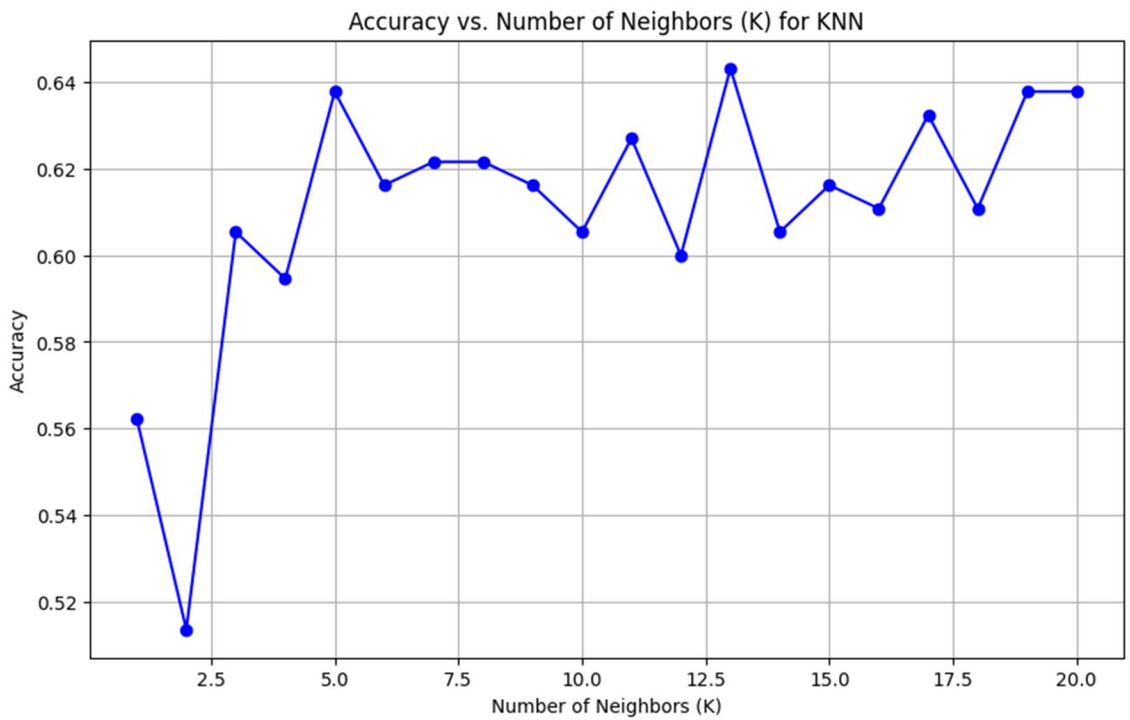
From the decision tree, we can observe that 'ApplicantIncome' and 'LoanAmount' are important features. Based on them the loan can be approved or rejected for an applicant.

- If LoanAmount is ≤ 80.5 and 111.5, the class is 5849.
- If LoanAmount is ≤ 143.0 and 163.0, the class is 3000.
- If LoanAmount is ≤ 217.5, the class is 2583 and further will be in the class 5417.

The decision tree uses the Gini impurity measure to make splits and determine class outcomes.

KNN

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.impute import SimpleImputer
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
import seaborn as sns
from sklearn.metrics import confusion_matrix


# Load the dataset
df = pd.read_csv('Loan-Approval-Prediction.csv')

# Select two features and target column
# Let's choose 'ApplicantIncome' and 'LoanAmount' as features, and 'Loan_Status' as the target
selected_features = df[['ApplicantIncome', 'LoanAmount']]
target_column = df['Loan_Status']

# Encode the target column (Y for Yes, N for No) to numerical values (1 for Yes, 0 for No)
le = LabelEncoder()
target_column_encoded = le.fit_transform(target_column)

# Handle missing values by imputing with the mean value of each column
imputer = SimpleImputer(strategy='mean')
selected_features_imputed = imputer.fit_transform(selected_features)

# Split the data into training and testing sets for KNN
X_train_knn, X_test_knn, y_train_knn, y_test_knn = train_test_split(selected_features_imputed, target_column_encoded, test_size=0.3, random_state=42)

# Initialize a list to store the accuracy scores for different values of k
accuracy_scores = []

# Try different values of k and compute accuracy for KNN
for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_knn, y_train_knn)
    accuracy = knn.score(X_test_knn, y_test_knn)
    accuracy_scores.append(accuracy)

# Plot the accuracy scores for different values of k for KNN
plt.figure(figsize=(10, 6))
plt.plot(range(1, 21), accuracy_scores, marker='o', linestyle='-', color='b')
plt.title('Accuracy vs. Number of Neighbors (K) for KNN')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.grid(True)
plt.show()
```


Accuracy vs. Number of Neighbors (K) for KNN
Accuracy
0.64
0.62
0.60
0.58
0.56
0.54
0.52
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Number of Neighbors (K)

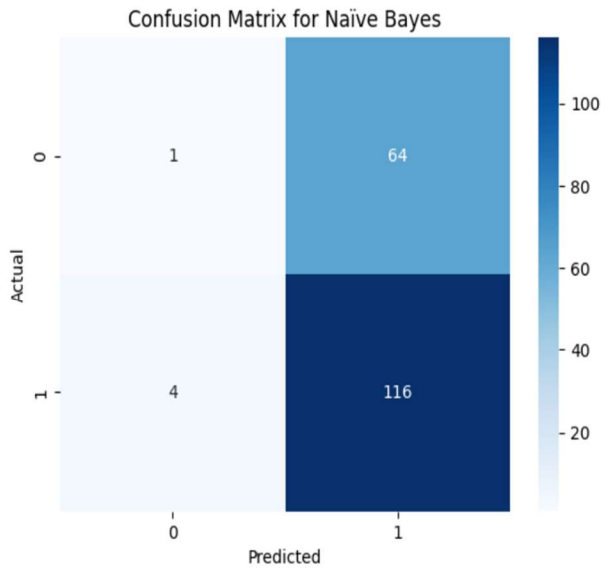
The graph displays the relationship between the number of neighbors (K) used in the K-Nearest Neighbors algorithm and the corresponding accuracy of loan approval predictions. The accuracy fluctuates as K increases, with a noticeable dip at K=2 and peaks around K=13 and K=18. The trend does not clearly indicate that increasing K leads to higher accuracy, suggesting that the optimal K value for this dataset might be around the local maxima points rather than a continuously higher K value.

According to the graph the optimal value of K for the K-Nearest Neighbors algorithm, based on the dataset, appears to be around 13 and 18, where the peaks of accuracy are observed. These points are achieves its highest accuracy, indicating they might be the best choices for the number of neighbors

Naïve Bayes

```
# Split the data into training and testing sets for Naïve Bayes
X_train_nb, X_test_nb, y_train_nb, y_test_nb = train_test_split(selected_features_imputed, target_column_encoded, test_size=0.3, random_state=42)

# Initialize and train the Naïve Bayes classifier
naive_bayes_classifier = GaussianNB()
naive_bayes_classifier.fit(X_train_nb, y_train_nb)

# Predict the test data for Naïve Bayes
y_pred_nb = naive_bayes_classifier.predict(X_test_nb)

# Evaluate the Naïve Bayes classifier
accuracy_nb = accuracy_score(y_test_nb, y_pred_nb)
classification_rep_nb = classification_report(y_test_nb, y_pred_nb)

print("Accuracy for Naïve Bayes:", accuracy_nb)
print("Classification Report for Naïve Bayes:\n", classification_rep_nb)
confusion = confusion_matrix(y_test_nb, y_pred_nb)
sns.heatmap(confusion, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix for Naïve Bayes")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
Accuracy for Naïve Bayes: 0.6324324324324324
Classification Report for Naïve Bayes:
              precision    recall  f1-score   support

           0       0.20      0.02      0.03        65
           1       0.64      0.97      0.77       120

    accuracy                           0.63       185
   macro avg       0.42      0.49      0.40       185
weighted avg       0.49      0.63      0.51       185
```



The classification report and confusion matrix for the Naïve Bayes classifier indicate that the model has an overall accuracy of about 63.24%. It performs well in correctly predicting the positive class (1) with a high recall of 0.97 but has a low precision for the negative class (0), indicating a high number of false positives. The confusion matrix shows that the model predicted the positive class correctly 116 times but had 64 false negatives, suggesting it is biased towards predicting the positive outcome.